



2025-2026(1) 面向对象程序设计 课外作业2 (考核)

学号: 202383290121 学院:
姓名: 梅应宝 专业:
班级: 23计科腾讯1班

00:57:31
交卷

单项选择题
试题数: 18
题型分: 100.0

试题列表: 题型数: 1, 试题数: 18, 总分: 100.0
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
试题要求(试题分数: 5分, 单项选择题 下第 5 题, 共 18 题)
下列各组类型声明符中, 含义相同的一组是 () 。

答题卡 上一题 下一题 标记试题

☐ A short int 和 int

☐ B unsigned short 和 short

☒ C signed short int 和 short

☐ D unsigned long int 和 long

请认真答题, 题型结束

-----请认真答题! -----

4【单选题】对C++编译器区分重载函数没有任何意义的信息是?

AI讲解

- A、 返回值类型
- B、 参数类型
- C、 关键字const
- D、 参数个数

我的答案: A



9.1 分

正确答案: A

8【多选题】已知函数定义如下，以下表述中，正确的有？

AI讲解

```
inline void swap(int &x, int &y)
{
    int t;

    t = x; x = y; y = t;
}
```

- A、 使用关键字inline修饰函数，表示将函数声明为内联函数。
- B、 理论上，将一个函数声明为内联函数，编译器会在编译期间将函数体嵌入到调用处。
- C、 理论上，将一个函数声明为内联函数，会减少函数调用和返回的时间与空间开销，提高程序执行效率。
- D、 在现代编译器中，没有使用inline修饰的函数，也有可能被编译为内联函数。而明确使用inline修饰的函数，则一定会被编译器编译为内联函数。

我的答案：ABC

✓ 9.1 分

正确答案：ABC

答案解析：inline只是语法层面的约定，要求编译器将函数作为内联函数编译。但是，编译器并不承诺一定会将inline修饰的函数作为内联函数。
通常，功能简单、规模较小又调用频繁的函数，适宜于声明为内联函数。但实际情况，依赖于编译器的处理。

- **前半句正确：**现代编译器（如GCC、Clang）会进行“自动内联优化”，即使函数没有 **inline** 修饰，只要编译器认为该函数短小、调用频繁，就可能将其优化为内联函数（例如类的成员函数默认可能被编译器内联）。
- **后半句错误：** **inline** 只是给编译器的一个“建议”（hint），而非“强制命令”。编译器有权忽略该建议：如果函数体过大、存在递归、包含复杂控制流（如循环、switch）等，即使加了 **inline**，编译器也可能拒绝内联（因为内联后反而会降低效率）。
因此，“明确使用 **inline** 的函数一定会被内联”的说法错误。

10 【多选题】设计者为游戏引擎开发玩家类Player，希望提供多种创建玩家对象的方式：既可以创建匿名玩家（使用默认名称），也可以创建具名玩家。设计者设计了两个方案。关于这两种设计方案，以下哪些说法是正确的？

方案1:

C/C++

```
1  class Player {
2  public:
3      Player() : m_name("Anonymous"), m_level(1) {} // 构造函数1
4      Player(const std::string& name) : m_name(name), m_level(1) {} // 构造函数2
5      Player(const std::string& name, int level) : m_name(name), m_level(level) {} // 构造函数3
6      // ... 其他成员 ...
7  private:
8      std::string m_name;
9      int m_level;
10 };
```

方案2:

C/C++

```
1  class Player {
2  public:
3      Player(const std::string& name = "Anonymous", int level = 1)
4          : m_name(name), m_level(level) {} // 单一构造函数
5      // ... 其他成员 ...
6  private:
7      std::string m_name;
8      int m_level;
9  };
```

- A、在方案1中，对象创建语句Player p1;Player p2("Alice");和Player p3("Bob", 5);分别合法地调用了构造函数1、2和3。
- B、如果未来需要增加一个仅接受int level参数（而不指定name）的构造函数（如Player(int level)），方案1可以通过增加一个新的重载构造函数来实现，而方案2无法实现这一需求。
- C、方案1和方案2都体现了OOP中“类”作为创建“对象”的蓝图这一基本概念，它们都定义了玩家对象应具有的属性（m_name, m_level）和初始化行为。
- D、方案2的代码更加简洁，减少了重复代码，因此在所有场景下都是比方案1更优的选择。
- E、在方案2中，对象语句Player p1;Player p2("Alice");和Player p3("Bob", 5);也同样都是合法的，并且创建出的对象与方案1中的效果完全相同。

我的答案：ABCE

✓ 10.0 分

正确答案：ABCE

答案解析：方案2在某些简单场景下是更简洁的选择，但方案1在灵活性、可扩展性和意图明确性方面有优势，具体选择取决于实际需求。

- **方案2的局限性：**方案2的构造函数为 `Player(const std::string& name = "Anonymous", int level = 1)`（双参数带默认值）。若新增 `Player(int level)`，当调用 `Player p(5);` 时，编译器会产生歧义：
 - 既可以匹配新的 `Player(int level)`（直接传int）；
 - 也可以匹配原构造函数（name 用默认值，level=5）。

编译器无法确定调用哪个构造函数，导致编译错误。因此方案2无法实现这一需求，B选项正确。

即在方案二中，若是再次重载`Player(int level)`,会与原本的构造函数冲突，因为原来的都是默认参数，分不清是name还是level

通常的拷贝初始化构造函数的参数是（ ）。

答题区

上一题

下一题

- ☐ A 对象的指针名
- ☒ B 对象的引用名
- ☐ C 对象的成员名
- ☐ D 对象名

默认访问方式是private! ! ! !

对于类中定义的成员，其隐含（默认）访问权限为（ ）。

答题区

上一题

下一题

- ☒ A private
- ☐ B static
- ☐ C protected
- ☐ D public

```
class A
{
    int a;
};
```

```
int main()
{
    A a;
    a.a = 1;
    cout << a.a << endl;
    return 0;
}
```

成员 "A::a" (已声明 所在行数:6) 不可访问 C/C++(265)

int A::a

生成 Copilot 摘要

查看问题 (Alt+F8) 快速修复... (Ctrl+.) 修复 (Ctrl+I)

结构体默认的是public

对于结构体中定义的成员，其隐含访问权限为（ ）。

答题区

上一题

下一题

- ☐ A static
- ☐ B protected
- ☐ C private
- ☒ D public